

Crestor Studio – projekt interfejsu z nakładkami, widgetami i asystentem AI

Organizacja interfejsu z zakładkami

Aby uczynić **Crestor Studio** maksymalnie przyjaznym i funkcjonalnym, zastosujemy układ z **zakładkami (tabs)**. Dzięki temu wszystkie narzędzia będą dostępne w jednym miejscu, podzielone na logiczne sekcje. Główne zakładki interfejsu mogą obejmować:

- **Nakładki (Overlays)** – konfiguracja elementów nakładanych na stream (np. kod QR, okno napiwków, standardowy widget).
- **Edycja profilu** – ustawienia profilu twórcy, w tym pasek postępu celu donacji i „ściana fanów”.
- **Pomoc (AI)** – interaktywna pomoc z wykorzystaniem asystenta AI, który poprowadzi krok po kroku przez konfigurację.

Każda zakładka odpowiada osobnej sekcji interfejsu. U góry strony umieszczamy poziome menu z nazwami zakładek. Kliknięcie w zakładkę wyświetli powiązane opcje, ukrywając pozostałe sekcje. Taki podział ułatwi mniej doświadczonym użytkownikom poruszanie się – widzą tylko to, nad czym aktualnie pracują.

Przykładowy kod HTML dla menu zakładek (styling Tailwind CSS, każda zakładka to link do identyfikatora sekcji):

```
<!-- Menu zakładek -->
<ul class="flex border-b bg-gray-50">
  <li class="mr-4">
    <a href="#overlays" class="inline-block py-2 px-4 font-semibold text-blue-600 border-b-2 border-blue-600">
      Nakładki
    </a>
  </li>
  <li class="mr-4">
    <a href="#profile" class="inline-block py-2 px-4 text-gray-600 hover:text-blue-600 hover:border-blue-600">
      Edycja profilu
    </a>
  </li>
  <li>
    <a href="#assistant" class="inline-block py-2 px-4 text-gray-600 hover:text-blue-600 hover:border-blue-600">
      Pomoc AI
    </a>
  </li>
</ul>
```

W powyższym kodzie pierwsza zakładka *Nakładki* jest podświetlona (niebieski tekst i dolna obwódka) jako aktywna. Pozostałe mają szary tekst i podświetlają się na niebiesko po najechaniu. **Tailwind CSS** umożliwia tu łatwe stylowanie: np. `border-b-2 border-blue-600` to dolna ramka aktywnej zakładki, a `hover:text-blue-600` dodaje efekt po najechaniu kursorem.

Każda sekcja poniżej menu powinna mieć odpowiadający `id` zgodny z linkiem zakładki (np. `id="overlays"`, `id="profile"`, `id="assistant"`). Dzięki temu proste skrypty mogą przełączać widoczność sekcji. Na początek możemy zastosować rozwiązanie czysto CSS/HTML – np. kotwice przewijające do sekcji – lub użyć JavaScript do dynamicznego przełączania (dodając/usuwając ukrycie klasą `hidden`). Dla czystości interfejsu zaleca się, by tylko jedna sekcja była widoczna na raz, reszta ukryta.

Sekcja Nakładki (Overlays)

Sekcja **Nakładki** pozwala twórcom tworzyć i konfigurować elementy wyświetlane na streamie jako nakładki. Każdy typ nakładki będzie miał osobny podrozdział ustawień. Dla klarowności, podzielimy tę sekcję na podsekcje (lub po prostu wyświetlimy ustawienia sekwencyjnie): **Nakładka QR (Kod QR)**, **Okno napiwków (Tip Modal)** oraz **Standardowy widget**. Użytkownik może dodawać nowe nakładki lub edytować istniejące; dla początkujących przewidzimy gotowe ustawienia domyślne na jedno kliknięcie – „**Wybierz domyślne**”, które automatycznie uzupełnią pola przykładowymi wartościami.

Nakładka QR (kod QR)

Nakładka QR wyświetla **kod QR** na ekranie, który widzowie mogą zeskanować (np. aby przejść do strony donacji, profilu twórcy lub innego linku). Każda nakładka jest tworzona jako odrębny element HTML osadzony w warstwie ponad tłem streama.

Element interfejsu: formularz pozwalający wgrać lub wygenerować kod QR oraz ustawić jego pozycję i rozmiar. Pola do konfiguracji mogą obejmować: URL do zakodowania, wybór koloru tła/przezroczystości, rozmiar kodu (w pikselach) oraz pozycję (np. wybór rogu ekranu).

Generowanie kodu QR: Można użyć biblioteki JS (np. `qrcode.js`) lub API do wygenerowania obrazka. Dla uproszczenia, w środowisku dev możemy generować kod QR online i zapisywać jako obraz PNG, a w produkcji zautomatyzować to. W UI możemy mieć przycisk „Generuj kod QR” – po wpisaniu URL, naciśnięciu przycisku pojawi się podgląd kodu QR.

Kod HTML nakładki QR (fragment):

```
<!-- Kontener nakładki QR (przykładowo w prawym dolnym rogu) -->
<div id="qrOverlay" class="fixed bottom-4 right-4 bg-white bg-opacity-80 p-2
rounded shadow-lg flex items-center">
  <!-- Obrazek kodu QR -->
  
  <!-- Opis/tekst opcjonalny obok kodu -->
  <span class="ml-2 text-gray-800 font-medium">Zeskanuj, aby wesprzeć</span>
</div>
```

W powyższym kodzie: kontener `div#qrOverlay` ma pozycjonowanie `fixed bottom-4 right-4`, dzięki czemu będzie zawsze 4 jednostki (1rem) od dołu i prawej krawędzi ekranu – to lokuje kod QR w prawym dolnym rogu. Klasy `bg-white bg-opacity-80 p-2 rounded shadow-lg flex items-center` dają półprzezroczyste białe tło, zaokrąglone rogi, cień oraz układ flexbox do wyśrodkowania zawartości pionowo. Wewnątrz jest `<img>` z kodem QR o stałym rozmiarze 96px (Tailwind `w-24 h-24`), bo 1w=4px w domyślnej konfiguracji) i ewentualny tekst obok.

Ustawienia w UI: Formularz może umożliwić zmianę pozycji poprzez wybór jednej z predefiniowanych opcji (np. 4 rogi ekranu) – odpowiada to zmianie klas Tailwind: `top-4 left-4`, `top-4 right-4`, `bottom-4 left-4`, `bottom-4 right-4`. Rozmiar kodu może zmieniać klasy `w-16 h-16` (mniejszy) lub `w-32 h-32` (większy) itp. Tekst opisu można edytować lub wyłączyć. Takie parametry będą zapisane dla danej nakładki i odzwierciedlone w wygenerowanym kodzie HTML.

Funkcjonalność: Po wygenerowaniu kodu QR, obrazek `#qrImage` powinien zostać zaktualizowany (np. poprzez ustawienie `src` na wygenerowany data URL lub ścieżkę do pliku). Można to osiągnąć skryptem w JavaScript, który wywołuje np. `QRGenerator.generate(url)` z biblioteki i ustawia wynik. Dodatkowo, jeśli chcemy umożliwić włączanie/wyłączanie nakładki w trakcie streamu, możemy dodać checkbox „Pokaż kod QR” – który toggluje widoczność kontenera (np. dodając/usuwając klasę `hidden` na `#qrOverlay`).

Okno napiwków (Tip Modal)

Okno napiwków to modalne okienko wyświetlane widzom, gdy chcą przekazać napiwek/donację. W **Creator Studio** twórca będzie mógł dostosować wygląd i teksty tego okna. Realizujemy je jako ukryty na co dzień element HTML, który pokazuje się (CSS `display` lub poprzez klasę Tailwind) dopiero po wywołaniu – np. po kliknięciu przycisku „Wesprzyj mnie” na streamie.

Element interfejsu: Formularz konfiguracji modala zawierający pola: tytuł okna (np. „Wesprzyj twórcę”), opis/zachęta do donacji, opcje kolorów (tło okna, kolor tekstu, przezroczystość tła strony za oknem), ewentualnie wybór pól formularza (czy ma być pole na wiadomość od fana, czy różne kwoty do wyboru). Twórca może podejrzeć wygląd modala na żywo w studio (podgląd).

Kod HTML modala napiwków:

```
<!-- Przycisk wywołujący modal (np. nakładka lub element na stronie) -->
<button onclick="openTipModal()" class="bg-blue-600 text-white px-4 py-2 rounded">
  Wesprzyj mnie
</button>

<!-- Overlay + Modal -->
<div id="tipModalOverlay" class="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center hidden">
  <!-- Modal okno -->
  <div class="bg-white text-gray-800 rounded-lg shadow-lg w-11/12 max-w-md p-6 relative">
    <!-- Przycisk zamknięcia -->
    <button onclick="closeTipModal()" class="absolute top-2 right-2 text-gray-500 hover:text-gray-800">&times;</button>
```

```

<!-- Treść modala -->
<h2 class="text-2xl font-semibold mb-4" id="tipModalTitle">Wesprzyj
mnie</h2>
<p class="mb-4" id="tipModalDescription">Dowolna kwota będzie ogromnym
wsparciem!</p>
<form id="tipForm">
  <label class="block mb-2">Kwota (PLN):
  <input type="number" min="1" class="w-full border px-3 py-2 rounded"
placeholder="10">
</label>
  <label class="block mb-4">Wiadomość (opcjonalna):
  <textarea class="w-full border px-3 py-2 rounded" rows="3"
placeholder="Napisz wiadomość..."></textarea>
</label>
  <button type="submit" class="bg-green-500 text-white w-full py-2
rounded hover:bg-green-600">Prześlą napiwek</button>
</form>
</div>
</div>

```

Powyższy kod przedstawia strukturę modala: całość osadzona jest w kontenerze `#tipModalOverlay`, który pokrywa cały ekran (`fixed inset-0`) i ma półprzezroczyste czarne tło (`bg-black bg-opacity-50`) pełniące rolę przyciemnionego tła strony, tzw. *backdrop*. Dodatkowo użyto `flex items-center justify-center`, aby modal był wycentrowany. Ten kontener domyślnie ma klasę `hidden`, by był niewidoczny.

Wewnątrz niego jest właściwe okno modala – `div` z białym tłem, zaokrąglonymi rogami i cieniem (`rounded-lg shadow-lg bg-white`), o maksymalnej szerokości `max-w-md` (średni ekran, ok. 28rem) i paddingu `p-6`. Użyliśmy też `relative`, by móc w nim absolutnie pozycjonować przycisk zamknięcia. Sam przycisk zamknięcia to `×` (krzyżyk) w prawym górnym rogu, a treść modala składa się z nagłówka, akapitu opisu i formularza. Formularz zawiera pola: **Kwota** (pole numeryczne) i **Wiadomość** (pole tekstowe) oraz przycisk „Prześlą napiwek”. Wszystkie elementy ostyleowano za pomocą Tailwind – np. pola mają ramkę i padding (`border px-3 py-2`) oraz zaokrąglenia. Przyciski mają wyróżniające kolory (`bg-blue-600` dla linku wywołującego, `bg-green-500` dla przesłania formularza).

JavaScript do otwierania/zamykania modala: W powyższym kodzie przyciski mają atrybuty `onclick` wywołujące funkcje `openTipModal()` i `closeTipModal()`. Te funkcje należy zdefiniować w pliku JS (np. `app.js`):

```

function openTipModal() {
  document.getElementById('tipModalOverlay').classList.remove('hidden');
}
function closeTipModal() {
  document.getElementById('tipModalOverlay').classList.add('hidden');
}

```

Takie proste funkcje dodają lub usuwają klasę `hidden`, co odpowiednio pokazuje lub chowa modal. Dodatkowo można dodać **animację** pojawiania się modala (np. dodając Tailwind classes `transition-`

`opacity duration-300` do overlay i `transform scale-105` zmieniające się przy otwarciu), aby UX był bardziej płynny.

Konfiguracja w UI Crestor Studio: Twórca powinien mieć możliwość edytować zawartość tekstową modala (np. tytuł, opis) – w kodzie reprezentowane przez elementy o id `tipModalTitle`, `tipModalDescription`. Możemy więc powiązać pola input w panelu edycji z aktualizacją tych elementów na podglądzie. Np. pole „Tytuł okna” aktualizuje `innerText` elementu `#tipModalTitle`. Podobnie kolory: np. wybór koloru tła modala może zmienić klasę `bg-white` na `bg-gray-800 text-white` (ciemny modal) – to też można zrobić dynamicznie przez JS lub generować różne klasy zależnie od ustawień.

Domyślne ustawienia: Dla mniej zaawansowanych, możemy przygotować **preset** – np. przycisk „Użyj domyślnego stylu”. Kliknięcie ustawi np. niebieskie tło nagłówka, doda ikonę, lub zastosuje sprawdzone wartości. To sprowadza się do ustawienia konkretnych klas i tekstów. Dzięki temu nawet niewprawny użytkownik otrzyma estetyczne okno bez zagłębiania się w szczegóły.

Standardowy widget (domyślny)

Standardowy widget to gotowy, uniwersalny element, który twórca może dodać jednym kliknięciem – przeznaczony „dla tych mniej rozgarniętych” lub chcących szybko wdrożyć podstawowe funkcje. Taki widget może zawierać np. kombinację kilku elementów: przycisk do donacji, licznik ostatniego subskrybenta, mały pasek postępu celu czy ikonki społecznościowe – w zależności od potrzeb platformy.

W Crestor Studio możemy to zrealizować jako **galerię predefiniowanych widgetów**. Na przykład, **Widget domyślny A:** prosty pasek z informacją „Wesprzyj streamera – kliknij tutaj”, **Widget domyślny B:** ramka z animowanym tekstem „Dziękuję za wsparcie!” pojawiająca się przy donacji, itp. Użytkownik wybiera widget z listy, a system dodaje go jako nakładkę.

Dla ilustracji, stwórzmy prosty **pasek u dołu ekranu** z wezwaniem do działania (CTA):

```
<!-- Standardowy widget: Pasek CTA na dole ekranu -->
<div id="defaultWidget" class="fixed bottom-0 left-0 w-full bg-purple-600
text-white text-center py-2">
  Wesprzyj mój kanał i dołącz do grona fanów! <button class="underline
font-bold" onclick="openTipModal()">Kliknij tutaj</button>
</div>
```

Ten widget to fioletowy pasek (kolor wyróżniający się, `bg-purple-600`) przyklejony do dołu (`fixed bottom-0 w-full`). Ma tekst zachęcający oraz przycisk/link, który otwiera okno napiwku (wykorzystując wcześniej zdefiniowaną funkcję `openTipModal()`). Emotikon dodaje przyjazny akcent. Twórca może w studio edytować treść tego paska (np. zmienić tekst, kolory) – albo zostawić domyślne.

W panelu konfiguracji wystarczy dla takiego widgetu prosty edytor tekstu i wybór kolorów tła/tekstu. Ponieważ jest to element „standardowy”, można ograniczyć możliwości edycji, by użytkownik go nie popsuł – np. tylko zmiana tekstu i kolorów, bez usuwania kluczowych składników (przycisku).

Strona techniczna: Dodanie tego widgetu może następować automatycznie po kliknięciu „Dodaj widget domyślny” – np. poprzez wstawienie powyższego kodu do DOM strony podglądu. Jeśli Crestor

Studio generuje konfigurację do osadzenia w streamie, ten kod trafi do finalnego pliku nakładki. Dobrze jest opatrzyć go komentarzami i legendą w kodzie, aby zaawansowani użytkownicy rozumieli jego działanie.

Sekcja Edycja profilu

Sekcja **Edycja profilu** zawiera elementy wyświetlane na profilu twórcy (np. stronie kanału), niezwiązane bezpośrednio z nakładkami na transmisję. Dwie kluczowe funkcje, które tu uwzględnimy, to **Pasek postępu celu finansowania** oraz „**Ściana fanów**”. Te elementy pozwalają budować zaangażowanie społeczności – pokazując postęp zbiorów i wyróżniając najwierniejszych fanów.

Pasek postępu (cel donacji)

Pasek postępu ilustruje graficznie postęp zbiórki pieniędzy na określony cel (np. „*Zbieram 1000 PLN na nowy mikrofon*”). To świetny sposób, by zmotywować widzów do wspierania – widzą ile już zebrano i ile brakuje.

Element interfejsu: formularz do ustawienia celu: nazwa/opis celu (np. „Nowy mikrofon”), docelowa kwota, opcjonalnie data zakończenia zbiórki. Twórca wpisuje te dane i zapisuje, a pasek postępu na jego profilu automatycznie się aktualizuje w oparciu o otrzymane donacje.

Implementacja paska postępu: Można go zaimplementować w HTML/CSS jako **prosty pasek wypełnienia**. Użyjemy zewnętrznego kontenera jako tła paska oraz wewnętrznego wypełnienia zmieniającego szerokość w zależności od procentu realizacji celu.

```
<!-- Kontener paska postępu celu -->
<div class="bg-gray-300 rounded-full h-6 w-full max-w-sm relative">
  <!-- Wypełnienie paska (dynamiczna szerokość) -->
  <div id="progressBarFill" class="bg-blue-500 h-6 rounded-full"
style="width: 40%;"></div>
  <!-- Tekst na pasku -->
  <span id="progressBarText" class="text-white font-bold text-sm absolute
inset-0 flex justify-center items-center">
    400 / 1000 PLN
  </span>
</div>
<p class="mt-2 text-gray-700 text-sm italic" id="progressBarLabel">Cel: nowy
mikrofon</p>
```

W powyższym kodzie zewnętrzny `div` to szare tło paska (`bg-gray-300`), o wysokości 1.5rem (`h-6`), pełnej szerokości maksymalnie 24rem (`max-w-sm`), z zaokrągleniem (`rounded-full`). Wewnątrz niego `div#progressBarFill` to niebieski pasek (`bg-blue-500`) o takiej samej wysokości i zaokrągleniu, którego **szerokość** ustawiamy poprzez styl inline `width: 40%;` (co odpowiada bieżącemu postępowi, tu 40% celu). Można to oczywiście ustawiać dynamicznie z JS – np. obliczyć procent = (zebrana_kwota / cel_kwota)100 i ustawić `.style.width = procent + '%'`. *Nad paskiem nałożyliśmy `span#progressBarText`, absolutnie pozycjonowany na cały pasek (`absolute inset-0 flex justify-center items-center` centrowany), z białym pogrubionym tekstem pokazującym "zebrano / cel PLN"*. Pod paskiem jest podpis celu `progressBarLabel` (np. „Cel: nowy mikrofon”).

Aktualizacja danych: W studio, po wpisaniu docelowej kwoty i aktualnie zebranej (można pozwolić też ręcznie ustawić ile już jest, lub pobierać z bazy danych automatycznie), skrypt wyliczy procent i zmieni szerokość `progressBarFill` oraz tekst. Np.:

```
function updateProgressBar(current, goal) {
  const percent = Math.min(100, (current/goal)*100);
  document.getElementById('progressBarFill').style.width = percent + '%';
  document.getElementById('progressBarText').innerText = `${current} / $
{goal} PLN`;
}
```

Ta funkcja zapewni, że pasek nie przekroczy 100% (użyliśmy `Math.min(100, ...)`). Należy ją wywołać przy inicjalizacji oraz po każdej zmianie danych celu. Jeśli cel ma datę zakończenia, można również wyświetlać odliczanie lub datę obok paska.

Stylizacja: Użyliśmy prostej jednolitej stylizacji. Twórca może w panelu wybrać inny kolor paska (wystarczy zmienić klasę np. na `bg-green-500` lub `bg-red-500`), a nawet styl (np. paski w stylu gradientu – to można dodać odpowiednią klasę Tailwind z pluginem lub niestandardowym CSS). Domyślnie dajemy neutralny niebieski.

Umiejscowienie: Na profilu użytkownika taki pasek może wyświetlać się u góry strony profilu albo w sekcji „O mnie”. Możemy dać możliwość wyboru wyrównania (lewo, środek, prawo) – co zmieni klasy Tailwind jak `mx-auto` (środek) lub korzystać z flexboxa do wyrównania.

Ściana fanów (Fan Wall)

„Ściana fanów” to miejsce, gdzie wyświetlamy najlepszych wspierających lub ostatnich donatorów – aby ich uhonorować. Może przybrać formę listy nazwisk/nicków, awatarów lub nawet wiadomości od fanów wyświetlanych publicznie.

Element interfejsu: W studio twórca może zarządzać listą fanów do wyświetlenia. Jeśli system automatycznie śledzi top donatorów czy subskrybentów, lista może być generowana automatycznie. W konfiguracji twórca może wybrać styl wyświetlania: np. **lista pionowa** (kolejne pozycje rankingowe) lub **siatka awatarów** (kilka kolumn z obrazkami). Można też ograniczyć ilu fanów pokazać (Top 5, Top 10 itp.).

Implementacja ściany fanów: Pokażemy przykład **siatki awatarów z podpisami**, łączący elastyczny układ Tailwind z dynamicznym wypełnianiem danych.

```
<!-- Ściana fanów (grid of avatars) -->
<div class="grid grid-cols-3 gap-4 max-w-sm">
  <!-- Przykładowy fan 1 -->
  <div class="text-center">
    
    <p class="text-sm mt-2 font-medium text-gray-800">FanNick1</p>
    <p class="text-xs text-gray-500">100 PLN</p>
  </div>
  <!-- Przykładowy fan 2 -->
```



```


![Fan 2](avatar2.jpg)

FanNick2



75 PLN



![Fan 3](avatar3.jpg)

FanNick3



50 PLN


```

Powyżej mamy kontener `grid` z trzema kolumnami (`grid-cols-3`) i odstępami `gap-4`. Każdy fan jest w osobnym `div` wyśrodkowanym (`text-center`). Pokazujemy obrazek avatara (okrągły dzięki `rounded-full`), rozmiar 4rem bo `w-16 h-16`, i wyśrodkowany poziomo przez `mx-auto`). Pod avatarom jest nick fana i kwota wsparcia mniejszym fontem. Możemy dodać np. 🏆 emoji dla top1 itp., według uznania stylizować.

Dynamika danych: W praktyce te dane (ścieżki do avatarów, nicki, kwoty) pochodziłyby z bazy danych (np. z listy top donatorów). Crestor Studio może pobierać je automatycznie. Jeśli jednak twórca chce ręcznie wyróżnić fanów, można dodać w panelu możliwość wpisania nicku i kwoty lub wgrania obrazka. Dla uproszczenia jednak zakładamy automatyczne uzupełnienie.

Alternatywy wyświetlania: Zamiast siatki, można przełączyć na **listę** – wtedy użyjemy np. `flex flex-col` i innego formatu (np. "1. FanNick1 - 100 PLN"). Można dać przełącznik trybu wyświetlania w ustawieniach. W CSS Tailwind zmiana jest prosta: dla listy pionowej z numeracją możemy użyć `list-decimal` i `list-inside` przy `<ol>` lub po prostu generować indeksy w tekście.

Umiejscowienie na profilu: „Ściana fanów” może znaleźć się w bocznej kolumnie profilu lub sekcji „Top Wspierający”. Ważne by była widoczna. W **Crestor Studio** możemy pokazać podgląd profilu z tym elementem.

Asystent AI – inteligentna pomoc krok po kroku

Jednym z najciekawszych elementów będzie wbudowany **asystent AI** (podobny do Google Gemini lub ChatGPT) pełniący rolę inteligentnej pomocy. Celem asystenta jest **prowadzenie użytkownika za rękę** podczas konfiguracji, odpowiadanie na pytania i sugerowanie rozwiązań – wszystko w ramach interfejsu Crestor Studio.

Formy integracji asystenta AI:

- **Chat kontekstowy:** W zakładce **Pomoc AI** (lub jako wysuwany widget dostępny na każdej stronie) umieścimy okno czatu, gdzie użytkownik może zadać pytanie, np. „*Jak dodać nakładkę z czatem na stream?*” lub „*Gdzie zmienić kolor paska donacji?*”. AI na podstawie dokumentacji i wiedzy udzieli odpowiedzi, np. wskaże konkretną opcję w menu lub fragment interfejsu.

- **Poradnik krok-po-kroku (tryb „Learn”)**: Można zaimplementować specjalny tryb, w którym AI poprowadzi użytkownika przez konfigurację komponentu. Przykładowo, użytkownik wybiera „Skonfiguruj pasek postępu z pomocą AI” – wtedy asystent np. wyświetla wskazówki: „*Krok 1: Przejdź do zakładki Edycja profilu i kliknij 'Dodaj pasek postępu'. (Zobacz podświetlenie na ekranie)*”. Tutaj asystent mógłby nawet *podświetlać* odpowiednie elementy interfejsu (to wymaga mechanizmu highlight – np. dodania klasy CSS z animacją obramowania do wskazanego przycisku). Po wykonaniu akcji użytkownik klika „Dalej” i AI przechodzi do następnej instrukcji, np. „*Krok 2: Wpisz docelową kwotę i naciśnij Zapisz*”, itd. Taki interaktywny tutorial mógłby korzystać z predefiniowanego scenariusza wspomaganego generatywnie (AI modyfikuje odpowiedzi zależnie od kontekstu, np. jeśli użytkownik wpisze 500 zamiast 1000, asystent uwzględni to w kolejnych krokach).
- **Sugestie i autokonfiguracja**: Asystent AI może także **podpowiadać ustawienia**. Np. po dodaniu nowej nakładki AI wyświetli komunikat: „*Chcesz szybko skonfigurować tę nakładkę? Mogę zastosować popularny styl.*” i zaproponuje automatycznie wypełnić niektóre pola (jak tekst, kolory) na podstawie preferencji użytkownika czy ogólnych trendów. Użytkownik może zaakceptować sugestie jednym kliknięciem. To coś w rodzaju inteligentnych „presetów” generowanych przez AI.

Techniczna realizacja AI w interfejsie: Za rozmowę AI będzie odpowiadał model językowy dostępny przez API. Można tu skorzystać np. z **Google Gemini API** (jeśli dostępny i odpowiada potrzebom) lub **OpenAI GPT-4 API**. Implementacja polega na wysyłaniu zapytań do API z kontekstem (np. pytanie użytkownika + ewentualnie fragment dokumentacji czy stan interfejsu) i wyświetlaniu odpowiedzi w oknie czatu.

- Interfejs czatu: możemy wykorzystać istniejące komponenty UI do chatbota albo napisać prosty: lista wiadomości (pytanie i odpowiedź) renderowana np. w `<div id="chatWindow">` z stylami bąbelków czatu. Nowe wiadomości dodajemy do DOM i przewijamy do dołu.
- Wysyłanie wiadomości: po wpisaniu pytania w polu input i naciśnięciu Enter, wywołujemy funkcję JS która:
- Doda pytanie do okna czatu (od strony użytkownika).
- Wyśle zapytanie do naszego serwera/backendu, który z kolei komunikuje się z API AI (Gemini/ChatGPT) i otrzyma odpowiedź.
- Po otrzymaniu odpowiedzi, wklei jej treść do okna czatu jako odpowiedź asystenta.
- **Kontent odpowiedzi** może zawierać nie tylko tekst, ale potencjalnie również **obrazy lub fragmenty kodu**. Np. użytkownik pyta „Pokaż kod przykładowego widgetu z licznikiem subskrybentów” – AI może wygenerować HTML/CSS, który pokażemy w stylu wstawki kodowej lub damy opcję „wstaw do projektu”.

Bezpieczeństwo i kontrola: Ważne, by asystent znał kontekst aplikacji. Dobrze jest wytrenować go lub w promptach systemowych przekazać mu słownik interfejsu (np. nazwy zakładek, dostępne akcje), aby jego odpowiedzi były precyzyjne i zgodne z rzeczywistością. Ponadto, ponieważ AI może halucynować, warto ograniczyć jego możliwości np. przez oferowanie mu bazy wiedzy (FAQ, dokumentacja) – można użyć techniki **Retrieval Augmented Generation**: nasz backend przy pytaniu użytkownika najpierw przeszuka dokumentację Crestor Studio, znajdzie pasujące informacje, i dołączy je do promptu dla AI. Dzięki temu AI odpowie na podstawie rzeczywistych danych, minimalizując błędy.

Przykład dialogu w interfejsie:

Użytkownik: „Jak dodać kod QR do streamu?”

Asystent: „Aby dodać nakładkę z kodem QR: 1) Przejdź do zakładki Nakładki. 2) Kliknij Dodaj nakładkę QR. 3) Wpisz URL, który ma być zakodowany i naciśnij Generuj QR. 4) Umieść wygenerowany kod w wybranym rogu

ekranu i zapisz zmiany. Na podglądzie streamu zobaczysz teraz kod QR do zeskanowania.” (Asystent mógłby tu dodatkowo pokazać strzałkę lub obramowanie na interfejsie wskazując przycisk *Dodaj nakładkę QR* – to wymaga integracji z front-end, np. event z AI do frontu z poleceniem highlight).

Taki **wirtualny tutor** znacząco obniży próg wejścia dla nowych użytkowników. Czują się oni zaopiekowani i szybciej skonfigurują swoje studio bez wertowania długiej dokumentacji.

Implementacja i struktura plików projektu

Aby wdrożyć powyższe elementy w kodzie, zachowując porządek, rozplanujmy strukturę plików oraz miejsca, gdzie trafią poszczególne fragmenty:

```
projekt/
├─ public/
│   ├─ index.html           # Główny plik HTML interfejsu Crestor Studio
│   (zakładki, sekcje)
│   ├─ styles.css           # Skompilowany CSS (Tailwind)
│   ├─ tailwind.config.js   # Konfiguracja Tailwind CSS (opcjonalnie, jeśli
│   customizujemy)
│   ├─ app.js               # Skrypt JavaScript obsługujący interakcje (tabs,
│   modale, AI chat)
│   └─ assets/              # Folder zasobów (obrazy avatarów, ikony,
│   wygenerowane kody QR itp.)
└─ (opcjonalnie foldery komponentów, jeśli używamy np. React/Vue)
```

Zakładamy tu prostą implementację bez frameworka, korzystając z czystego HTML + Tailwind + vanilla JS. Jeśli projekt jest oparty o framework (np. React), wówczas każdy z wymienionych elementów byłby komponentem w folderze src (np. `Overlays.jsx`, `ProfileEditor.jsx`, `AiAssistant.jsx`), ale koncepcja pozostaje podobna.

1. index.html: Będzie zawierał **szkielet interfejsu** – menu zakładek oraz kontenery sekcji. Każda sekcja (Nakładki, Edycja profilu, Pomoc) możemy opakować w `<section>` z odpowiednim `id`. Wewnątrz umieścimy podsekcje/elementy opisane wcześniej.

Przykład (fragmenty):

```
<body class="bg-white text-gray-800 font-sans">
  <!-- Nawigacja zakładek -->
  <nav>
    ... (menu zakładek z listy, jak pokazano wyżej) ...
  </nav>

  <!-- Sekcja Nakładki -->
  <section id="overlays" class="p-4">
    <h1 class="text-xl font-bold mb-4">Nakładki</h1>
    <!-- Formularz dodawania nowej nakładki -->
    <button class="bg-green-600 text-white px-3 py-1 rounded"
      onclick="addNewOverlay()">+ Dodaj nakładkę</button>
```

```

<!-- Podsekcja: Nakładka QR -->
<div class="mt-6 border-t pt-4">
  <h2 class="text-lg font-semibold mb-2">Nakładka QR (kod QR)</h2>
  <!-- Ustawienia kodu QR: pole URL, podgląd, pozycja -->
  URL: <input type="text" id="qrUrlInput" class="border px-2 py-1
rounded" placeholder="https://...">
  <button class="bg-gray-200 px-2 py-1 rounded" onclick="generateQR()">Generuj
QR</button>
  <!-- ...więcej pól... -->
  <!-- Podgląd nakładki QR -->
  <div class="relative mt-4">
    <iframe id="qrPreviewFrame" class="border w-full h-64"
src="qr_preview.html"></iframe>
    <!-- Zakładamy, że generujemy podgląd w osobnym pliku/iframe dla
izolacji stylów -->
  </div>
</div>

<!-- Podsekcja: Okno napiwków -->
<div class="mt-6 border-t pt-4">
  <h2 class="text-lg font-semibold mb-2">Okno napiwków (Tip Modal)</h2>
  <!-- Pola konfiguracji modala: tytuł, opis, kolory -->
  Tytuł: <input type="text" id="tipTitleInput" class="border px-2 py-1
rounded" value="Wesprzyj mnie"><br>
  Opis: <input type="text" id="tipDescInput" class="border px-2 py-1
rounded w-full" value="Dowolna kwota..."><br>
  Kolor tła: <input type="color" id="tipBgColor" value="#ffffff">
  Kolor tekstu: <input type="color" id="tipTextColor"
value="#000000"><br>
  <button class="mt-2 bg-gray-200 px-3 py-1 rounded"
onclick="resetTipModalDefaults()">Ustaw domyślne</button>
  <!-- Podgląd modala -->
  <div class="mt-4">
    <button class="bg-blue-600 text-white px-4 py-2 rounded"
onclick="openTipModal()">Podgląd okna napiwków</button>
  </div>
</div>

<!-- Podsekcja: Standardowy widget -->
<div class="mt-6 border-t pt-4">
  <h2 class="text-lg font-semibold mb-2">Standardowy widget</h2>
  <!-- Lista dostępnych presetów widgetów -->
  <div class="grid grid-cols-2 gap-4">
    <div class="border p-2">
      <h3 class="font-medium">Domyślny pasek CTA</h3>
      <p class="text-sm">Pasek na dole z przyciskiem „Wesprzyj mnie”.</p>
      <button class="bg-purple-600 text-white text-sm px-2 py-1 mt-2
rounded" onclick="addDefaultWidget('cta_bar')">
        Dodaj widget
      </button>
    </div>
  </div>
</div>

```

```

        </div>
        <!-- Możliwe inne widgety domyślne w kolejnych kolumnach... -->
    </div>
    <!-- Podgląd widgetu domyślnego (jeśli dodany) -->
    <div id="defaultWidgetPreview" class="mt-4"></div>
</div>
</section>

<!-- Sekcja Edycja profilu -->
<section id="profile" class="p-4 hidden">
    <h1 class="text-xl font-bold mb-4">Edycja profilu</h1>
    <!-- Pasek postępu -->
    <div class="mb-6">
        <h2 class="text-lg font-semibold mb-2">Pasek postępu celu</h2>
        Cel: <input type="text" id="goalName" class="border px-2 py-1 rounded"
placeholder="np. nowy mikrofon"><br>
        Kwota docelowa: <input type="number" id="goalAmount"
class="border px-2 py-1 rounded" placeholder="1000"><br>
        Obecnie zebrano: <input type="number" id="currentAmount" class="border
px-2 py-1 rounded" placeholder="0"><br>
        <button class="bg-gray-200 px-3 py-1 mt-2 rounded"
onclick="updateProgress()">Zapisz i aktualizuj</button>
        <!-- Podgląd paska postępu -->
        <div class="mt-4">
            <!-- Wstawiamy tu kod paska z wcześniej zdefiniowanego komponentu -->
            <div id="goalBarContainer">
                <!-- (treść paska postępu z divami, jak zdefiniowano wcześniej) -->
            </div>
        </div>
    </div>
    <!-- Ściana fanów -->
    <div>
        <h2 class="text-lg font-semibold mb-2">Ściana fanów</h2>
        Tryb wyświetlania:
        <select id="fanWallMode" class="border px-1 py-1 rounded">
            <option value="grid">Siatka awatarów</option>
            <option value="list">Lista rankingowa</option>
        </select>
        <button class="bg-gray-200 px-3 py-1 ml-2 rounded"
onclick="refreshFanWall()">Odśwież</button>
        <!-- Podgląd ściany fanów -->
        <div id="fanWallPreview" class="mt-4">

<!-- (domyślnie wstawiamy tu wygenerowany HTML ściany fanów, np. grid z top 3
jak wyżej) -->
        </div>
    </div>
</section>

<!-- Sekcja Pomoc AI -->
<section id="assistant" class="p-4 hidden">

```

```

<h1 class="text-xl font-bold mb-4">Pomoc AI</h1>
<div id="chatWindow" class="border rounded p-2 h-64 overflow-y-scroll bg-gray-50 text-sm">
  <!-- Wiadomości czatu będą się tu pojawiać -->
  <div class="text-gray-600 mb-2"> <em>W czym mogę pomóc? Zadaaj pytanie, a poprowadzę Cię krok po kroku.</em></div>
</div>
<div class="mt-2 flex">
  <input type="text" id="chatInput" class="border flex-1 px-3 py-1 rounded-l" placeholder="Zapytaj AI...">
  <button class="bg-blue-600 text-white px-4 py-1 rounded-r" onclick="sendQuestion()">Wyślij</button>
</div>
</section>

<script src="app.js"></script>
</body>

```

(Powyższy kod jest rozbudowanym przykładem – w praktyce może zostać podzielony na mniejsze części. Ważne jest jednak, aby pokazać w którym miejscu co wstawić i jak nazwać elementy.)

2. styles.css: Zawiera skompilowany Tailwind CSS. W trakcie developmentu można użyć CDN (np. `<link href="https://cdn.tailwindcss.com" rel="stylesheet">` co generuje style na podstawie `data-tailwind-config`). Dla produkcji lepiej jednak przygotować własną kompilację, by usunąć nieużywane klasy (*purge*). W pliku `tailwind.config.js` możemy zdefiniować np. dodatkowe kolory (`smoke` dla tła modala jak w tutorialu) czy własne utility jeśli potrzebujemy.

3. app.js: Skrypt obsługujący interaktywność. Powinien zawierać:

- **Logikę przełączania zakładek:** np. event listener na linki w menu, który zapobiega domyślnej nawigacji i pokazuje odpowiednią sekcję. Przykład:

```

document.querySelectorAll('nav a[href^="#"]').forEach(link => {
  link.addEventListener('click', evt => {
    evt.preventDefault();
    // Ukryj wszystkie sekcje
    document.querySelectorAll('section').forEach(sec =>
    sec.classList.add('hidden'));
    // Usuń aktywną klasę z wszystkich zakładek
    document.querySelectorAll('nav a').forEach(a =>
    a.classList.remove('text-blue-600', 'border-blue-600'));
    // Pokaż docelową sekcję
    const targetId = link.getAttribute('href');
    document.querySelector(targetId).classList.remove('hidden');
    // Oznacz klikniętą zakładkę jako aktywną
    link.classList.add('text-blue-600', 'border-b-2', 'border-blue-600');
  });
});

```

Powyższy kod iteruje po wszystkich linkach nawigacji i dodaje obsługę kliknięcia. Ukrywa wszystkie `<section>` (dodaje klasę `hidden` Tailwind, która ma `display: none`), usuwa wyróżnienie z wszystkich zakładek, następnie pokazuje wybraną sekcję i wyróżnia kliknięty link (dodając mu klasy tekstu i obramowania). Dzięki temu mamy **jednostronicowy interfejs** reagujący na menu.

- **Funkcje dodawania/konfiguracji nakładek:** np. `addNewOverlay()` może dodawać nowy panel konfiguracji kolejnej nakładki w sekcji (jeśli pozwalamy na wiele instancji). W naszym uproszczonym przypadku mieliśmy statycznie jedną instancję każdego typu, więc niekoniecznie potrzebujemy dodawać duplikaty – ewentualnie można to rozbudować.

- **Funkcje obsługujące podglądy:**

- `generateQR()` – wygeneruje obrazek kodu QR. Może np. użyć dostępnego API: pobrać wartość z `#qrUrlInput`, zrobić `fetch("https://api.qrserver.com/v1/create-qr-code/?size=150x150&data=" + encodeURIComponent(url))`, a następnie ustawić `document.getElementById('qrImage').src` na otrzymany URL (ta usługa generuje kod QR w locie). Następnie pokazać `#qrOverlay` w ramce podglądu. Ewentualnie wygenerować wewnątrz `<canvas>` jak używamy biblioteki.
- Aktualizacja **Tip Modal**:
 - `resetTipModalDefaults()` – przywraca domyślne wartości pól (np. tytuł "Wesprzyj mnie", kolor tła biały itp.).
 - Można również na bieżąco nasłuchiwać zmian pól `tipTitleInput`, `tipDescInput` i aktualizować zawartość podglądu modala (np. `document.getElementById('tipModalTitle').innerText = titleInput.value`).
 - Kolory tła/tekstu: może być tricky jeśli chcemy podgląd w tym samym elemencie. Można jednak bezpośrednio modyfikować style modala: `document.querySelector('#tipModalOverlay .bg-white').style.backgroundColor = chosenColor;` lub dodać/zmienić klasę Tailwind (to drugie trudniejsze dynamicznie bez regeneracji stylów, więc prościej inline styles dla podglądu).
- `addDefaultWidget(type)`: dla naszego `type='cta_bar'` dodaje kod domyślnego widgetu do `#defaultWidgetPreview` lub bezpośrednio do DOM strony. W przykładzie z CTA wystarczy:

```
function addDefaultWidget(type) {
  if(type === 'cta_bar') {
    document.getElementById('defaultWidgetPreview').innerHTML =

`<div class="fixed bottom-0 left-0 w-full bg-purple-600 text-white text-center py-2">
  Wesprzyj mój kanał... <button class="underline font-bold"
onclick="openTipModal()">Kliknij tutaj</button>
</div>`;
  }
}
```

Potem można pokazać komunikat, że widget dodany. (W praktyce pewnie widżety domyślne będą dodawane do listy nakładek również, by streamer mógł je umieścić w obs, ale to zależy od systemu).

- `updateProgress()`: wywołuje wcześniej wspomnianą `updateProgressBar(current, goal)` używając wartości z pól `#goalAmount` i `#currentAmount`. Uaktualnia też etykietę `#progressBarLabel` na „Cel: ...” z pola `#goalName`.
- `refreshFanWall()`: w zależności od trybu `fanWallMode` generuje odpowiedni HTML listy lub gridu i wstawia do `#fanWallPreview`. Jeśli mamy dane (np. w tablicy `topFans = [{nick, amount, avatar}, ...]`), to tutaj przez JS można iterować i budować string HTML. W trybie `grid` format jak powyżej, w trybie `list` można zrobić:

```
let html = '<ol class="list-decimal list-inside">';
topFans.forEach(fan => {
  html += '<li>${fan.nick} - ${fan.amount} PLN</li>';
});
html += '</ol>';
fanWallPreview.innerHTML = html;
```

Dodatkowo, jeśli tryb `grid`, użyć layoutu z obrazkami jak wcześniej (tu też można iterować i wstawiać). W naszym prototypie można po prostu wstawić statyczny HTML z trzema fanami (jak w kodzie wyżej) jako przykład.

• Integracja AI czatu:

- `sendQuestion()`: pobiera tekst z `#chatInput`, wyświetla go w `#chatWindow`, czyści input, następnie wysyła zapytanie do API (to wymaga endpointu – np. wystawionego w backend). Ponieważ jesteśmy w kontekście frontendu, można użyć `fetch` do własnego serwera (np. `fetch('/askAI', { method: 'POST', body: JSON.stringify({question: pytanie}) })`). Back-end (nieopisany tu) odeśle odpowiedź AI, którą odbierzemy `.then(res => res.json()).then(data => { ... })` i wstawimy do `#chatWindow` jako kolejny `<div>` z odpowiedzią (np. poprzedzając tekstem " ").
- Można dodać pewne ozdoby, np. animowane kropki w trakcie oczekiwania (wstaw tymczasową wiadomość "..."), a po otrzymaniu zamienić ją na pełną odpowiedź.
- Warto też przewinąć kontener czatu na dół po dodaniu nowej wiadomości: `chatWindow.scrollTop = chatWindow.scrollHeight`.

4. Inne pliki: - `qr_preview.html` (opcjonalnie jak w kodzie umieściliśmy `<iframe>` dla podglądu QR) – może zawierać minimalny HTML z osadzonym aktualnie generowanym obrazkiem. To izoluje styl nakładki od stylów panelu konfiguracyjnego. Ewentualnie można to robić bez `iframe`, w samym panelu. - **assets/** – trzymamy tu np. domyślne avatary, wygenerowane obrazy kodów QR (jeśli nie generujemy na żywo), czy ikonki używane w UI (np. ikonka AI, jeśli dodamy zamiast tekstu "Pomoc AI" symbol asystenta).

Podsumowanie i legenda

Powyżej przedstawiłem kompletny plan zaprojektowania **Crestor Studio** – od struktury interfejsu po szczegóły implementacji w plikach. Jako specjalista front-end upewniłem się, że rozwiązanie jest **użyteczne i przyjazne**: interfejs jest intuicyjny (zakładki, sekcje, opisy), **Tailwind CSS** zapewnia spójny styl i ułatwia wdrożenie produkcyjne, a integracja **AI** wnosi innowacyjny sposób wsparcia użytkownika.

Legenda / wyjaśnienie skrótów i ścieżek:

- `fixed`, `absolute`, `relative` – pozycjonowanie CSS elementów (ważne dla nakładek i modali).
- `inset-0`, `top-4`, `right-4`, etc. – klasy Tailwind oznaczające odsunięcie od krawędzi (użyte przy pozycjonowaniu QR i modala).
- `bg-...` i `text-...` – klasy tła i tekstu (np. `bg-blue-600` niebieskie tło, `text-white` biały tekst). W projekcie możemy zmieniać je dla personalizacji kolorów.
- `rounded`, `rounded-full` – zaokrąglenie rogów (pełne koło dla avatarów).
- `shadow-lg` – standardowy duży cień (użyty dla modala i nakładki QR dla lepszego odcięcia od tła).
- `mx-auto`, `text-center`, `flex`, `items-center`, `justify-center` – ułatwiają centrowanie elementów odpowiednio.
- **Pliki:**
 - **HTML (index.html)** – zawiera strukturę zakładek i sekcji.
 - **CSS (styles.css)** – styl aplikacji, głównie Tailwind (można dodać własne style jeśli potrzebne).
 - **JS (app.js)** – logika interakcji (przełączanie zakładek, otwieranie modali, aktualizacje podglądu, komunikacja z AI).
 - **Assets** – obrazy i inne zasoby jak wygenerowane QR, avatary fanów, itp.
- Przyciski typu **Dodaj** w UI wywołują funkcje JS, które tworzą odpowiednie elementy (np. dodają HTML do podglądu nakładki lub inicjują generowanie kodu QR). Ścieżka kodu została opisana przy każdym takim przypadku.
- Gdy zobaczysz w powyższym opisie notację (...) lub `<!-- ... -->`, oznacza to miejsce, gdzie trzeba wstawić kod lub gdzie następuje pewna logika – starałem się to jasno skomentować w przykładach.

Projektując w ten sposób, otrzymamy przejrzyste **Creator Studio** z wyraźnie wydzielonymi częściami. Dzięki zastosowaniu gotowych stylów Tailwind, komponenty będą responsywne i estetyczne od razu, co jest ważne „pod produkcję”. Mniej doświadczeni użytkownicy skorzystają z domyślnych widgetów i pomocy AI, a zaawansowani docenią możliwość personalizacji każdego elementu.

Mam nadzieję, że ten szczegółowy przewodnik – z opisem **co, w jakim pliku, jak uzupełnić i podpiąć** – pozwoli sprawnie zaimplementować Crestor Studio od strony frontendu. Powodzenia w wdrożeniu!
